

Improving GNSS Positioning using Neural Network-based Corrections

Ashwin V. Kanhere*, Shubh Gupta*, Akshay Shetty and Grace Gao
Stanford University

BIOGRAPHY

Ashwin V. Kanhere is a Ph.D. candidate in the Department of Aeronautics and Astronautics at Stanford University. He received an M.S. in Aerospace Engineering from the University of Illinois at Urbana-Champaign in 2019 and a B.Tech in Aerospace Engineering from the Indian Institute of Technology Bombay, Mumbai in 2017. His research interests are in reliable navigation, with a particular interest in the fusion of Computer Vision and LiDAR applications.

Shubh Gupta is a Ph.D. candidate in the Department of Electrical Engineering at Stanford University. He received his M.S. degree in Electrical Engineering from Stanford University in 2020 and a B.Tech degree in Electrical Engineering with a minor in Computer Science and Engineering from the Indian Institute of Technology Kanpur in 2018. His research interests include safe positioning and perception for autonomous navigation.

Akshay Shetty is a postdoctoral researcher in the Department of Aeronautics and Astronautics at Stanford University. He obtained his Ph.D. in Aerospace Engineering from University of Illinois at Urbana-Champaign, and his B.Tech. in Aerospace Engineering from Indian Institute of Technology Bombay. His research interests include safe positioning and navigation of autonomous systems.

Grace Gao is an assistant professor in the Department of Aeronautics and Astronautics at Stanford University. Before joining Stanford University, she was an assistant professor at University of Illinois at Urbana-Champaign. She obtained her Ph.D. degree at Stanford University. Her research is on robust and secure positioning, navigation and timing with applications to manned and unmanned aerial vehicles, robotics, and power systems.

ABSTRACT

Deep Neural Networks (DNNs) are a promising tool for Global Navigation Satellite System (GNSS) positioning in the presence of **multipath and non-line-of-sight errors**, owing to their ability to model complex errors using data. However, developing a DNN for GNSS positioning presents various challenges, such as 1) **poor numerical conditioning caused by large variations in measurements and position values across the globe**, 2) **varying number and order within the set of measurements due to changing satellite visibility**, and 3) **overfitting to available data**. In this work, we address the aforementioned challenges and propose an approach for GNSS positioning by applying **DNN-based corrections to an initial position guess**. Our DNN learns to output the **position correction using the set of pseudorange residuals and satellite line-of-sight vectors as inputs**. The limited variation in these input and output values improves the numerical conditioning for our DNN. We design our DNN architecture to combine information from the available GNSS measurements, which vary both in number and order, by leveraging recent advancements in set-based deep learning methods. Furthermore, **we present a data augmentation strategy for reducing overfitting in the DNN by randomizing the initial position guesses**. We first perform simulations and show an improvement in the initial positioning error when our DNN-based corrections are applied. After this, **we demonstrate that our approach outperforms a WLS baseline on real-world data**. Our implementation is available at github.com/Stanford-NavLab/deep_gnss.

I. INTRODUCTION

In the last decade, deep learning has been applied in several localization applications involving complex and high-dimensional sensor inputs, such as camera images and LiDAR pointclouds (Choy et al., 2020; Kendall et al., 2015; Wang et al., 2017). Deep learning algorithms utilize labelled data to 1) **discover an effective representation, or *embedding*, of the sensor inputs needed for localization**, and to 2) build an approximate model, represented by a deep neural network (DNN), of the sensor input-position output relationship. Since both the embeddings and the model are learned using data, these methods have shown better performance than analytical methods when sensor inputs are affected by environmental factors, such as occlusions and dynamic obstacles (Sünderhauf et al., 2018).

Given the success of deep learning in localization using sensor inputs, it is natural to consider applying deep learning for

* These authors contributed equally to this work

localization using GNSS measurements. This is especially important for localization in urban and semi-urban environments, where multipath and non-line-of-sight (NLOS) effects add environment-dependent additive biases to GNSS measurements, which are challenging to model analytically. The error distributions in GNSS measurements due to these effects are often non-Gaussian, which reduces the accuracy of traditional techniques that rely on Gaussian approximations of the error (Reisdorf et al., 2016; Wen et al., 2020; Zhu et al., 2018). Since DNNs can learn the relationship between the measurements and corresponding positions using data, they offer a promising alternative for localization in urban and semi-urban environments.

Availability of labelled datasets containing ground truth positions is necessary for training a DNN for localization. The recent increase in public datasets containing GNSS pseudorange measurements along with the associated ground truth positions is promising for the development of deep learning algorithms for GNSS-based localization (Fu et al., 2020). These datasets are collected over different driving scenarios, such as highway, urban and semi-urban, and under different operating conditions. Thus, these datasets provide a variety of input-output pairs for training the DNN.

Although labelled data with GNSS pseudorange measurements is becoming increasingly available, three main challenges must be addressed before this data can be used to train a DNN for localization:

1. **Different variations in values of the GNSS data.** Satellite positions in the earth-centered, earth-fixed (ECEF) frame of reference can take values between $[-20200, 20200]$ km in all three axes with variations of the same magnitude. On the other hand, GNSS pseudorange measurements have values of around 20200 km but variations on a much smaller scale, of about 100 m. Similarly, GNSS receiver positions in the ECEF reference frame take values approximately between $[-6000, 6000]$ km in all three axes with variations of the same magnitude. The large difference in the ratio of meaningful variations to received values causes the optimization problem of training a DNN to be numerically ill-conditioned, resulting in large changes to the DNN's parameters at each update and numerical instability (Goodfellow et al., 2016; J. McKeown et al., 1997). Furthermore, naïvely rescaling the satellite position and pseudorange measurement values risks loss of information necessary for positioning due to finite precision of floating point operations. Therefore, additional strategies for representing the satellite positions and pseudorange measurements must be considered.
2. **Varying number and order of GNSS measurements.** Since the number of visible satellites at a measurement epoch depends on the environment, the set of measurements received at different epochs often contains different number of GNSS signals. Additionally, for the same set of measurements, the output of GNSS-based localization algorithms should be independent of the order of measurements within the set. However, most DNN architectures are designed for a fixed number of inputs supplied in a pre-determined order, requiring the use of specialized architectures for GNSS-based localization (Lee et al., 2019; Skianis et al., 2020; Zaheer et al., 2017).
3. **Limitation in collecting vast amounts of real-world GNSS data and ground truth.** Collection of large-scale GNSS datasets for deep learning is limited by the need of ground truth positions associated with the measurements, which requires sophisticated hardware. Therefore, the existing GNSS datasets with ground truth are collected at a few locations in the world and at specific times. These datasets are limited both in the geography and in the variety of observed pairs of GNSS measurements and positions. For instance, the ECEF positions of both the receiver and the satellites captured in a dataset collected within California will not include the ECEF positions seen in a dataset collected within Australia. Using such limited data in deep learning often results in DNN models that overfit on the training data and perform poorly on unseen inputs (Goodfellow et al., 2016).

In this work, we address these challenges and develop a deep learning algorithm for localization using GNSS pseudorange measurements. We propose converting the position estimation problem solved by traditional GNSS positioning algorithms into the problem of estimating position corrections to an initial position guess. In our approach, we use a DNN to learn a functional mapping from GNSS measurements to these position corrections, as illustrated in Fig. 1. This paper is based on our work in (Kanhare, Gupta, Shetty, & Gao, 2021).

The main contributions of our work are:

1. Design a DNN to estimate position corrections to an initial position guess. To our knowledge, our approach is one of the first to use a DNN with outputs directly in the GNSS positioning domain.
2. Use a set-based DNN architecture to handle the varying number and order of GNSS inputs at each measurement epoch.
3. Use numerically conditioned inputs and outputs, in a local frame of reference, for the DNN. We use residuals and line-of-sight (LOS) vectors as inputs along with position correction outputs in the local North-East-Down (NED) frame of reference for numerically stable training and to encourage global applicability of the algorithm.
4. Develop a geometry-based data augmentation strategy to prevent overfitting in the DNN and improve its generalization to new GNSS measurements. Our strategy generates new data points for training the DNN by leveraging the geometrical relationship between randomized initial position guesses, residuals, LOS vectors and position corrections.

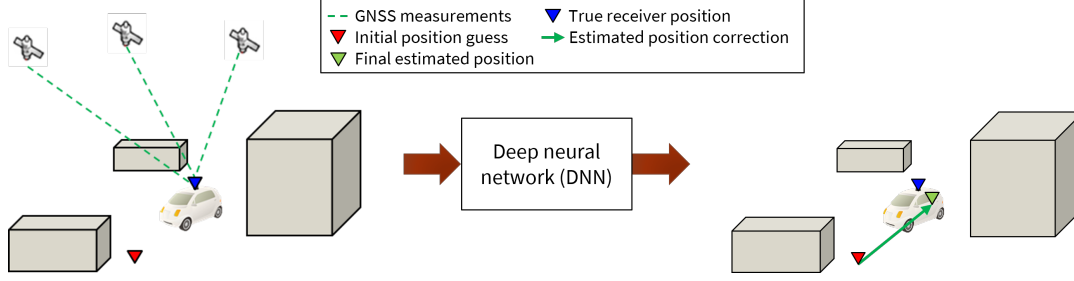


Figure 1: Our approach for applying deep learning for GNSS-based localization. Given GNSS pseudorange measurements and satellite positions, our method uses a DNN to estimate position corrections to an initial position guess.

5. **Validate our proposed approach on simulations and real-world data from the Android Raw GNSS Measurements Dataset (Fu et al., 2020).**

Our implementation is also publicly available at github.com/Stanford-NavLab/deep_gnss.

The rest of this paper is organized as follows. Section II. discusses related work; Section III. gives relevant background on set-based deep learning; Section IV. provides a description of our proposed method including details of numerical conditioning of the DNN input-output values, our data augmentation strategy and the neural network architecture; Section V. details our experimental validation on both simulated and real-world datasets. Finally, Section VI. concludes this paper.

II. RELATED WORK

Previous work has primarily used deep learning in the GNSS measurement domain for detecting faulty measurements or estimating pseudorange uncertainty. In (Hsu, 2017), the author proposes an approach that uses a Support Vector Machine (SVM) for detection of multipath, LOS and NLOS measurements. The SVM is given a feature vector consisting of pseudorange residuals, pseudorange rate residuals, and signal-to-noise ratio for each measurement. The author shows that the SVM improves the NLOS, LOS and multipath detection rate compared to a fixed detection threshold. In (Munin et al., 2020), the authors detect multipath signals using a Convolutional Neural Network (CNN) by learning relevant visual features from the receiver correlator output for each satellite measurement. In (Zhang et al., 2021), the authors use a combination of Long Short-Term Memory (LSTM) and CNNs to predict satellite visibility and pseudorange uncertainty. The LSTM architecture proposed by the authors handles varying number and order of GNSS measurements in detecting multipath signals. However, these previous works (Hsu, 2017; Munin et al., 2020; Zhang et al., 2021) focus on applying deep learning in the GNSS measurement domain and not directly in the GNSS positioning domain.

In line with our proposed approach, several previous works have proposed estimating the pose (position and orientation) from sensor measurements by estimating and applying a correction to an initial pose guess. In (Cattaneo et al., 2019), the authors propose a localization approach using a camera image measurement and a LiDAR map of the environment. The approach trains several DNNs to iteratively correct an initial pose guess based on a learned disparity between the camera image and an expected image constructed from the LiDAR map. In (Peretroukhin & Kelly, 2018) the authors generate correction factors within a Factor Graph using pairwise image measurements from a camera. The correction factor is obtained from a DNN and represents the relative pose between the two pairs of images. Although the idea of estimating position using corrections to an initial guess has been explored in literature, it has not been applied to the problem of GNSS-based positioning using deep learning, which is the focus of this work.

III. BACKGROUND: DEEP LEARNING ON SETS

Since the visibility of different satellites changes depending on both the location and the time of measurement, GNSS positioning output must be consistent for inputs containing different number and ordering of the measurements. For example, the position estimated using GNSS measurements from satellites numbered 1 – 8 must be similar to that estimated using satellites numbered 5 – 10, even if both the number of measurements and the order in which measurements from the same satellites appear are different in both cases. These inputs of varying size and order are commonly referred to as "set-valued inputs". Set-valued inputs pose unique challenges to common DNN architectures, which are designed to operate on inputs with fixed dimensions and are sensitive to the order in which different elements appear within the input (Zaheer et al., 2017).

Recently, DNN architectures that can handle set-valued inputs have been explored in literature (Lee et al., 2019; Skianis et al., 2020; Zaheer et al., 2017). For set-valued inputs comprised of elements in domain $\mathcal{X}$ and outputs in domain $\mathcal{Y}$, the objective

of these DNN architectures is to learn a function $\Phi : 2^{\bar{\mathcal{X}}} \rightarrow \bar{\mathcal{Y}}$, such that

$$\mathbf{Y}^{(i)} = \Phi \left(\mathcal{X}^{(i)} \right), \quad (1)$$

$$\mathcal{X}^{(i)} = \{\mathbf{X}_1^{(i)}, \mathbf{X}_2^{(i)}, \dots, \mathbf{X}_{M^{(i)}}^{(i)}\} \quad \forall M^{(i)} \in \mathbb{N}, \quad (2)$$

where $2^{\bar{\mathcal{X}}}$ denotes the power set containing all combinations of elements with domain $\bar{\mathcal{X}}$; $\mathcal{X}^{(i)}$ denotes the i th set-valued data instance with $\mathbf{X}_1^{(i)}, \mathbf{X}_2^{(i)}, \dots, \mathbf{X}_{M^{(i)}}^{(i)} \in \bar{\mathcal{X}}$; $\mathbf{Y}^{(i)} \in \bar{\mathcal{Y}}$ denotes the i th set-valued output; and $M^{(i)}$ is the number of elements in $\mathcal{X}^{(i)}$, which can vary across data instances.

To operate on sets, Φ satisfies the following two properties

1. **Order invariance.** For an input $\mathcal{X} = \{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_M\}$ and its permutation $\mathcal{X}' = \{\mathbf{X}_{\pi(1)}, \mathbf{X}_{\pi(2)}, \dots, \mathbf{X}_{\pi(M)}\}$, which has the same elements as $\mathcal{X}$ but with a different order defined by the operator $\pi(\cdot)$, the function output should remain the same, i.e., $f(\mathcal{X}) = f(\mathcal{X}')$.
2. **Consistency with variable input size.** For inputs $\mathcal{X} = \{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_M\}$ and $\mathcal{X}' = \{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_{M'}\}$, with different number of elements ($M \neq M'$), f has well-defined outputs i.e. $f(\mathcal{X}), f(\mathcal{X}') \in \mathcal{Y}$.

DNNs equipped to handle set-valued inputs realize these properties in three main process steps: 1) generating input embeddings, 2) aggregating these embeddings and 3) processing the aggregated embeddings to produce the output (Soelch et al., 2019). In the following description of DNNs for set-valued inputs, we walk through these three steps for applying Φ to a single data instance. Correspondingly, we simplify the notation from $\mathcal{X}^{(i)}$ to $\mathcal{X}$.

In the first step, an encoder network Φ_{encoder} , composed of feed-forward neural network layers individually processes each element $\mathbf{X}_m \forall m \in \{1, \dots, M\}$ within the set-valued input $\mathcal{X}$ to obtain corresponding feature embeddings $\mathbf{f}_m$ such that

$$\mathbf{f}_m = \Phi_{\text{encoder}}(\mathbf{X}_m). \quad (3)$$

For the set input, we denote this encoding process as

$$\mathcal{F} = \Phi_{\text{encoder}}(\mathcal{X}), \quad (4)$$

where $\mathcal{F} = \{\mathbf{f}_1, \dots, \mathbf{f}_M\}$ is the set of all embeddings such that $\mathbf{f}_m = \Phi_{\text{encoder}}(\mathbf{X}_m)$.

In the second step, the aggregation function combines the embeddings $\mathbf{f}_m$ into a fixed-size aggregated embedding $\mathbf{e}$ of the inputs using an aggregation function $\Phi_{\text{aggregate}}$

$$\mathbf{e} = \Phi_{\text{aggregate}}(\mathcal{F}). \quad (5)$$

Since the aggregation function $\Phi_{\text{aggregate}}$ combines the embeddings from different input elements in the set to a fixed-size output, $\Phi_{\text{aggregate}}$ can be chosen such that it is number and order invariant.

Finally, in the third step, a decoder network Φ_{decoder} composed of feed-forward neural network layers processes the embedding $\mathbf{e}$ to produce the output $\mathbf{Y}$

$$\mathbf{Y} = \Phi_{\text{decoder}}(\mathbf{e}). \quad (6)$$

As a result of the three steps, the overall function $\Phi : 2^{\bar{\mathcal{X}}} \rightarrow \bar{\mathcal{Y}}$ can be represented as

$$\mathbf{Y} = \Phi(\mathcal{X}) = \Phi_{\text{decoder}}(\Phi_{\text{aggregate}}(\Phi_{\text{encoder}}(\mathcal{X}))) \quad (7)$$

If the aggregation function $\Phi_{\text{aggregate}}$ is chosen to be number and order invariant, the composite function Φ is both invariant to the ordering of the inputs and unaffected by the number of elements. A variety of aggregations $\Phi_{\text{aggregate}}$ that fulfill this criteria have been studied in literature, such as [sum, max-pooling, and learned aggregations](#) (Soelch et al., 2019).

Set transformer (Lee et al., 2019) is a particular type of DNN architecture for set-valued inputs that uses learned aggregations to construct the fixed-size input encoding $\mathbf{e}$. In set transformers, the learned aggregations consider interactions between different set elements while combining the embeddings $\mathbf{f}_m$. Modeling these element to element interactions has shown to perform well in tasks such as clustering, where the effective aggregation needs to be determined from the set elements themselves. Furthermore, these learned aggregations have been shown to perform well for a wide range of hyperparameters (Soelch et al., 2019).

GNSS-based localization benefits from such considerations in modeling element-element interactions, since comparisons between different GNSS measurements aid in the detection of multipath and NLOS errors (Mikhailov & Nikandrov, 2012;

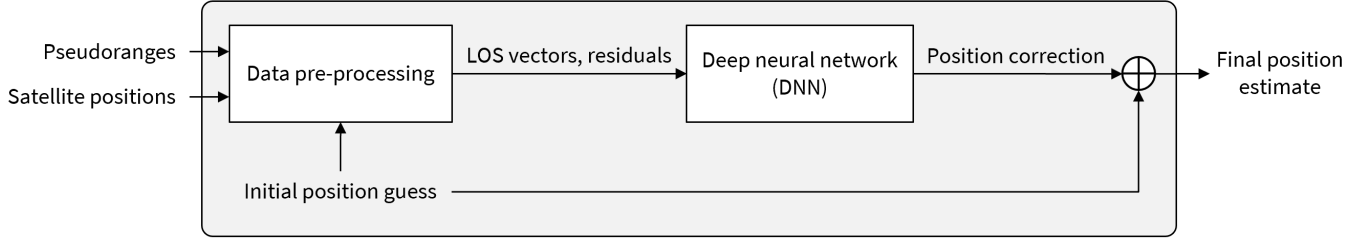


Figure 2: Overall architecture. First, we use GNSS pseudorange measurements and satellite positions to generate pairs of pseudorange residuals and LOS vectors with respect to an initial position guess as explained in Section IV.1. Next, we train a DNN (Section IV.2) to estimate position corrections to this initial position guess. Here while training the DNN we incorporate strategies explained in Section IV.3 to improve its generalization capabilities. Finally, the position correction from our DNN is used to improve the initial position guess and obtain our final position estimate (Section IV.4).

Savas & Dovis, 2019). Additionally, the set transformer aggregation function $\Phi_{\text{aggregate}}$ is number and order invariant which allows its application to set-valued inputs, such as GNSS measurements. Hence, we employ the set transformer within our DNN architecture to handle set-valued GNSS measurements.

IV. PROPOSED METHOD

In this section, we describe our approach for developing a DNN for estimating corrections, to an initial position guess, using GNSS pseudorange measurements. First, we formulate the problem of estimating position corrections with data values that are numerically well-conditioned for deep learning. Then, we describe the architecture and training process of our DNN that employs a set transformer to process the set-valued inputs derived from GNSS measurements and estimates the position correction. Next, we explain our strategies to overcome the problems of geographic sparsity of data and overfitting. Finally, we illustrate our inference procedure for a new set of GNSS measurements. Fig. 2 shows the overall architecture of our algorithm.

1. Position Correction from GNSS Measurements

At a measurement epoch, typical methods estimate the position using GNSS pseudorange measurements $\rho_1^{(i)}, \rho_2^{(i)}, \dots, \rho_{M^{(i)}}^{(i)}$, collected from a position $\mathbf{p}_{\text{ECEF}}^{(i)}$ in the ECEF frame of reference. Here, i denotes the i th data instance within the dataset of GNSS measurements with $M^{(i)}$ satellites visible from $\mathbf{p}_{\text{ECEF}}^{(i)}$. The position $\mathbf{P}_m^{(i)}$ of the satellite associated with each m th measurement is available from the ephemerides broadcast by the International GPS Service (IGS) ground stations. For inputs to typical GNSS localization algorithms, consider the set $\bar{\mathcal{M}}^{(i)}$ of pairs of pseudoranges with corresponding satellite positions, defined as

$$\bar{\mathcal{M}}^{(i)} = \left\{ \left(\rho_1^{(i)}, \mathbf{P}_1^{(i)} \right), \left(\rho_2^{(i)}, \mathbf{P}_2^{(i)} \right), \dots, \left(\rho_{M^{(i)}}^{(i)}, \mathbf{P}_{M^{(i)}}^{(i)} \right) \right\}. \quad (8)$$

In a machine learning setting, the objective is to learn a function $\tilde{\Phi}$ that outputs an estimate $\hat{\mathbf{p}}_{\text{ECEF}}^{(i)}$ of the position $\mathbf{p}_{\text{ECEF}}^{(i)}$ using the input $\bar{\mathcal{M}}^{(i)}$, for the i th data instance, where N is total number of instances in the dataset,

$$\hat{\mathbf{p}}_{\text{ECEF}}^{(i)} = \tilde{\Phi} \left(\bar{\mathcal{M}}^{(i)} \right) \quad \forall i \in \{1, \dots, N\}. \quad (9)$$

As previously mentioned in Sec. I., Eq. (9) has poor numerical conditioning for deep learning due to large variations in the input and output values. Therefore, we rewrite Eq. (9) as the addition of an estimated position correction $\Delta \hat{\mathbf{p}}_{\text{ECEF}}^{(i)}$ to an initial position guess $\hat{\mathbf{p}}_{\text{init}}^{(i)}$

$$\hat{\mathbf{p}}_{\text{ECEF}}^{(i)} = \hat{\mathbf{p}}_{\text{init}}^{(i)} + \Delta \hat{\mathbf{p}}_{\text{ECEF}}^{(i)}. \quad (10)$$

In this new setting, the objective is to learn a function $\tilde{\Phi}$ that outputs an estimate of the position correction $\Delta \hat{\mathbf{p}}_{\text{ECEF}}^{(i)}$ using the input $\bar{\mathcal{M}}^{(i)}$, such that

$$\Delta \hat{\mathbf{p}}_{\text{ECEF}}^{(i)} = \tilde{\Phi} \left(\bar{\mathcal{M}}^{(i)}, \hat{\mathbf{p}}_{\text{init}}^{(i)} \right) \quad \forall i \in \{1, \dots, N\}. \quad (11)$$

If $\hat{\mathbf{p}}_{\text{init}}^{(i)}$ is selected in the vicinity of the true position $\mathbf{p}_{\text{ECEF}}^{(i)}$, the different values of the true position correction $\Delta\mathbf{p}_{\text{ECEF}}^{(i)} = \mathbf{p}_{\text{ECEF}}^{(i)} - \hat{\mathbf{p}}_{\text{init}}^{(i)}$ are restricted to a small range, as opposed to the large variations in $\mathbf{p}_{\text{ECEF}}^{(i)}$, resulting in better conditioning of the output for function $\tilde{\Phi}$.

To improve the conditioning of the input variables, we consider the pseudorange model (Morton et al., 2021)

$$\rho_m^{(i)} = \left\| \mathbf{P}_m^{(i)} - (\hat{\mathbf{p}}_{\text{init}}^{(i)} + \Delta\mathbf{p}_{\text{ECEF}}^{(i)}) \right\| + \varepsilon_m^{(i)}, \quad (12)$$

where $\varepsilon_m^{(i)}$ denotes the error in the measurement due to both factors that can be modelled, such as satellite and receiver clock drift and atmospheric effects, as well as factors that are difficult to model analytically, such as effects of multipath and NLOS signals.

Assuming $\left\| \Delta\mathbf{p}_{\text{ECEF}}^{(i)} \right\| \ll \left\| \mathbf{P}_m^{(i)} - \hat{\mathbf{p}}_{\text{init}}^{(i)} \right\|$, we linearize the expression in Eq. (12) about $\hat{\mathbf{p}}_{\text{init}}^{(i)}$

$$\rho_m^{(i)} - \left\| \mathbf{P}_m^{(i)} - \hat{\mathbf{p}}_{\text{init}}^{(i)} \right\| \approx \nabla \left\| \mathbf{P}_m^{(i)} - (\hat{\mathbf{p}}_{\text{init}}^{(i)} + \Delta\mathbf{p}) \right\|_{\Delta\mathbf{p}=0} \cdot \Delta\mathbf{p}_{\text{ECEF}}^{(i)} + \varepsilon_m^{(i)}, \quad (13)$$

$$\implies r_m^{(i)} \approx -\mathbf{1}_m^{(i)} \cdot \Delta\mathbf{p}_{\text{ECEF}}^{(i)} + \varepsilon_m^{(i)}, \quad (14)$$

where $r_m^{(i)}$ denotes the pseudorange residual (difference between $\rho_m^{(i)}$ and expected pseudorange at $\hat{\mathbf{p}}_{\text{init}}^{(i)}$) for the m th satellite and $\mathbf{1}_m^{(i)}$ denotes the LOS vector for the m th satellite from $\hat{\mathbf{p}}_{\text{init}}^{(i)}$, both for the i th data instance. Thus, given an initial guess $\hat{\mathbf{p}}_{\text{init}}^{(i)}$, we convert the set of received measurements $\mathcal{M}$ into a set containing pairs of residuals and corresponding LOS vectors, defined as

$$\mathcal{M}^{(i)} = \left\{ \left(r_1^{(i)}, \mathbf{1}_1^{(i)} \right), \left(r_2^{(i)}, \mathbf{1}_2^{(i)} \right), \dots, \left(r_{M^{(i)}}^{(i)}, \mathbf{1}_{M^{(i)}}^{(i)} \right) \right\}, \quad (15)$$

which is equivalently represented as

$$\mathcal{M}^{(i)} = \left(\mathcal{R}^{(i)}, \mathcal{I}^{(i)} \right), \quad (16)$$

where $\mathcal{R}^{(i)} = \{r_1^{(i)}, \dots, r_{M^{(i)}}^{(i)}\}$ and $\mathcal{I}^{(i)} = \{\mathbf{1}_1^{(i)}, \dots, \mathbf{1}_{M^{(i)}}^{(i)}\}$.

Assuming $\hat{\mathbf{p}}_{\text{init}}^{(i)}$ is in the vicinity of the true position, the residuals $r_m^{(i)}$ typically vary in the range of tens of meters while the LOS vectors $\mathbf{1}_m^{(i)}$ are unit constrained. Hence, the measurement set $\mathcal{M}$, defined in Eq. (16) is a better conditioned input to the DNN than the received measurement set $\tilde{\mathcal{M}}$.

As a result of the input and output conditioning, the effective objective of the DNN in our approach is to learn a functional mapping Φ such that

$$\Delta\hat{\mathbf{p}}_{\text{ECEF}}^{(i)} = \Phi(\mathcal{M}^{(i)}) \quad \forall i \in \{1, \dots, N\}. \quad (17)$$

To summarize, the input to the network is a set $\mathcal{M}^{(i)}$ of residuals with corresponding LOS vectors and the output of the network is the position correction $\Delta\hat{\mathbf{p}}^{(i)}$ at the i th sample of the dataset.

2. DNN for Estimating Position Corrections

To obtain the estimated position corrections $\Delta\hat{\mathbf{p}}_{\text{ECEF}}^{(i)}$ from the conditioned set-valued inputs $\mathcal{M}^{(i)}$ using Eq. 17, we develop a neural network based on the set transformer (Lee et al., 2019) architecture discussed in Section III.

Our DNN architecture comprises of four components which we train together to estimate the position corrections from input set $\mathcal{M}$ of residuals and LOS vectors. First, as a part of the encoder Φ_{encoder} , a fully connected network, with ReLU activation functions, generates a high-dimensional embedding of each input, comprising of a residual r from $\mathcal{R}$ and the associated LOS vector $\mathbf{1}$ from $\mathcal{I}$. Each embedding is a D -dimensional vector and is an instance of a measurement in the latent space. Here D is a hyper-parameter of the network architecture and can be different after encoding or after aggregation in the network. In this work, we choose $D = 64$ as the hyperparameter throughout the network. Then, a set transformer encoder based on the set transformer encoder block (Lee et al., 2019) further refines the embeddings by modeling interactions between different set elements. Next, a network for learned aggregation $\Phi_{\text{aggregate}}$, based on set transformer pooling block (Lee et al., 2019), determines the influence of each set element on the position correction output and combines the embeddings based on these influences. Finally, a set transformer decoder network Φ_{decoder} , composed of a set transformer decoder and a linear layer (Goodfellow et al., 2016),

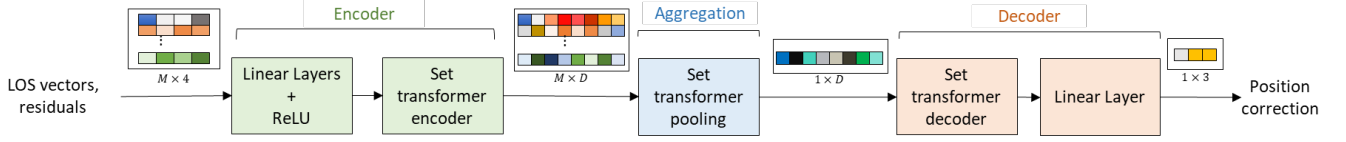


Figure 3: Our DNN architecture. The network consists of three steps: the encoder, the aggregation and the decoder. First, the LOS vectors and the pseudorange residuals from M satellites are concatenated into a $M \times 4$ matrix. The encoder, comprising of fully connected layers and the set transformer encoder block (Lee et al., 2019), then produces a $1 \times D$ embedding for each satellite, outputting a $M \times D$ matrix. Throughout the network, D is a hyperparameter for the dimension of the latent space that inputs are projected to and feature embeddings are computed in. Next, the aggregation function, or the set transformer pooling block (Lee et al., 2019), combines these embeddings into a fixed-size embedding and outputs a $1 \times D$ array. Finally, the decoder outputs the 3-dimensional position corrections.

processes the aggregated embedding to determine the position correction output $\Delta \hat{\mathbf{p}}_{\text{ECEF}}$. Section III. briefly explains the set transformer encoder, aggregation and decoder blocks. Fig. 3 depicts the DNN architecture for our proposed approach.

We train the DNN by minimizing the mean-squared-error (MSE) between a batch of the estimated and the true corrections as the loss function

$$\text{MSE} = \frac{1}{N_b} \sum_{j=1}^{N_b} \left\| \Delta \mathbf{p}_{\text{ECEF}}^{(j)} - \Delta \hat{\mathbf{p}}_{\text{ECEF}}^{(j)} \right\|^2, \quad (18)$$

where N_b is the batch size.

3. Strategies For DNN Generalization

While a DNN trained from Eq. 17 has access to well-conditioned inputs and outputs, its ability to generalize to new data instances is limited by 1) the geographic sparsity of the data, and 2) variations in inputs and outputs encountered during training. In this subsection, we present strategies to overcome these limitations and improve the generalization capability of the DNN.

a) Overcoming Geographic Sparsity by Change of Reference Frame

Geographic sparsity arises because the dataset is collected at fixed regions on the globe. The satellite LOS vectors and position corrections in the data collected in one part of the world may significantly differ from that in data from another part, resulting in measurements from some regions being disproportionately represented in the dataset. This disproportionality increases the difficulty in training the DNN to accurately estimate corrections for positions all around the globe, since certain input-output relations might be missing from the dataset.

To incentivize the DNN to generalize to inputs from across the globe, we make the input-output behavior independent of the location the data was collected in. We achieve this by changing the frame of reference of the inputs and outputs from the global ECEF frame to the local North-East-Down (NED) frame about $\mathbf{p}_{\text{init}}^{(i)}$. In the inputs, the residuals remain the same while the LOS vectors change. We represent this transformation as

$$\mathcal{T}_{\text{ECEF} \rightarrow \text{NED}}^{(i)} \left(\mathcal{M}^{(i)} \right) = \left(\mathcal{R}^{(i)}, \mathcal{T}_{\text{ECEF} \rightarrow \text{NED}}^{(i)} \left(\mathcal{I}^{(i)} \right) \right), \quad (19)$$

where $\mathcal{R}^{(i)}$ and $\mathcal{I}^{(i)}$ are defined the same as in Eq. (16) and $\mathcal{T}_{\text{ECEF} \rightarrow \text{NED}}^{(i)}$ denotes the transformation from the ECEF frame to the NED frame.

The neural network estimates $\Delta \hat{\mathbf{p}}_{\text{ECEF}}^{(i)}$ by estimating $\Delta \hat{\mathbf{p}}_{\text{NED}}^{(i)}$ which gets modified according to the transformation

$$\Delta \hat{\mathbf{p}}_{\text{ECEF}}^{(i)} = \mathcal{T}_{\text{NED} \rightarrow \text{ECEF}}^{(i)} \left(\Delta \hat{\mathbf{p}}_{\text{NED}}^{(i)} \right), \quad (20)$$

$$\Delta \hat{\mathbf{p}}_{\text{NED}}^{(i)} = \Phi \left(\mathcal{T}_{\text{ECEF} \rightarrow \text{NED}}^{(i)} \left(\mathcal{M}^{(i)} \right) \right) \quad \forall i \in \{1, \dots, N\}, \quad (21)$$

where $\mathcal{T}_{\text{NED} \rightarrow \text{ECEF}}^{(i)}$ denotes the transformation from the NED frame to the ECEF frame.

The above mentioned transformation changes the inputs and outputs to a local frame of reference. The modified inputs and outputs make our proposed architecture amenable to measurements from parts of the world that were not seen during the training process, as measurements are similar to the network's training data after transformation.

b) Increasing Data Variation using Geometry-based Data Augmentation

Using limited data to train a DNN with several parameters often leads to overfitting, where the DNN memorizes input-output pairs specific to the training dataset (Goodfellow et al., 2016). Data augmentation is a commonly used technique to reduce overfitting which introduces new data points to the DNN during training by transforming existing training samples based on the problem context.

We introduce a geometry-based data augmentation strategy for training a DNN to estimate position corrections from pseudorange measurement residuals and LOS vectors. Alg. 1 illustrates the process for generating new data points from a data instance. Our augmentation strategy leverages the geometric aspect of GNSS-based positioning by changing the value of the initial position guess $\hat{\mathbf{p}}_{\text{init}}^{(i)}$ each training epoch to generate new residuals $\mathcal{R}^{(i)}$, LOS vectors $\mathcal{I}^{(i)}$ and corrections $\Delta \mathbf{p}_{\text{ECEF}}^{(i)}$ via Eq. (14). New initial position guesses are generated by adding zero-mean uniformly distributed noise to the ground truth position $\mathbf{p}^{(i)}$. As a result, new samples are generated without any correlation, thus regularizing the training process and allowing the network to better learn the input-output mapping relationship. Finally, the network sees new samples in every training epoch, which prevents it from overfitting on the training data.

4. Inference

In this section, we illustrate our process to use the trained DNN for estimating the position $\hat{\mathbf{p}}_{\text{ECEF}}$ from new GNSS pseudorange measurements and the corresponding satellite positions, represented by the set $\mathcal{M}$.

First, we obtain an initial position guess $\mathbf{p}_{\text{init}}$, from a traditional localization algorithm or using prior knowledge, that we assume is in the vicinity of the true position $\mathbf{p}_{\text{ECEF}}$. Then, we use Eq. (17) to determine the input set $\mathcal{M}$ that comprises of pseudorange residuals $\mathcal{R}$ and corresponding LOS vectors $\mathcal{I}$ in the NED reference frame with respect to $\mathbf{p}_{\text{init}}$. Using the set $\mathcal{M}$ as an input to the DNN, we evaluate the position correction in the NED frame $\Delta \hat{\mathbf{p}}_{\text{NED}}$ and convert it to the position correction in the ECEF frame $\Delta \hat{\mathbf{p}}_{\text{ECEF}}$. Finally, we add the correction $\Delta \hat{\mathbf{p}}_{\text{ECEF}}$ to $\mathbf{p}_{\text{init}}$ to obtain the position estimate $\hat{\mathbf{p}}_{\text{ECEF}}$ using

$$\hat{\mathbf{p}}_{\text{ECEF}} = \mathbf{p}_{\text{init}} + \Delta \hat{\mathbf{p}}_{\text{ECEF}}. \quad (22)$$

V. EXPERIMENTS

We validate our approach using a **simulated dataset and real-world measurements from the Android Raw GNSS Measurements Dataset (Fu et al., 2020)**. We use simulations to verify the performance of our network in a setting with controlled measurement errors and access to precise ground truth information. In the validation on real-world data, we compare the accuracy of our proposed approach to that of weighted least squares (WLS) (Morton et al., 2021), **which is an equivalent traditional localization algorithm and serves as a baseline comparison**. In experiments on both data types, we used the same network architecture, optimizer parameters, data generalization method, and other experimental hyperparameters. These parameters are described in Section V. 1., followed by experimental evaluation on the simulated dataset in Section V. 2. and evaluation on the Android Raw GNSS Measurements Dataset in Section V. 3.

1. Experimental Parameters

Algorithm 1: Geometry-based data augmentation

Input: Set $\mathcal{M}$ of paired pseudorange measurements and satellite positions and ground truth position $\mathbf{p}_{\text{ECEF}}$

Parameters: Number of augmented data points K and vector-valued initialization range η

Output: A list of residuals $\bar{R}$, LOS vectors $\bar{I}$, and position corrections $\Delta \bar{P}$

```

1  $\bar{R} \leftarrow [], \bar{I} \leftarrow [], \Delta \bar{P} \leftarrow []$ 
2 for  $k \leftarrow 1$  to  $K$  do
3   Sample  $\mathbf{p}_{\text{init}}$  uniformly from  $[\mathbf{p}_{\text{ECEF}} - \eta, \mathbf{p}_{\text{ECEF}} + \eta]$ 
4   Generate  $\mathcal{R}, \mathcal{I}, \Delta \mathbf{p}_{\text{ECEF}}$  from  $\mathcal{M}$ , and  $\mathbf{p}_{\text{init}}$  using Eq. (14)
5   Assign values  $\bar{R}[k] \leftarrow \mathcal{R}, \bar{I}[k] \leftarrow \mathcal{I}, \Delta \bar{P}[k] \leftarrow \Delta \mathbf{p}_{\text{ECEF}}$ 
6 return  $\bar{R}, \bar{I}, \Delta \bar{P}$ 

```

In our experiments, a fully trained network occupies 611kB on disk for 151, 107 parameters. We use an instance of the network described in Section IV. 2. where the inputs (residuals and LOS vectors) are projected into a latent space of dimension $D = 64$ by a linear layer, followed by a ReLU activation (Goodfellow et al., 2016). In our implementation, we chose $D = 64$ as the dimension of the latent spaces in which all projected and embedded features exist.

The projected features are then encoded by two Transformer Encoder layers (Vaswani et al., 2017) that operate on the features sequentially. The encoded features are pooled using a pooling attention module (Lee et al., 2019), which is followed by two sequential Transformer Decoder layers and a linear layer to output the 3D position correction. We do not use batch normalization or dropout techniques at any point in the network architecture.

Our experiments are performed with data batches of 64 samples and the network is trained for 200 epochs. The DNN parameters are optimized using Adam (Kingma & Ba, 2014) with a learning rate $\alpha = 3 \times 10^{-4}$ and moving window average $\beta_1 = 0.9$, and $\beta_2 = 0.99$.

At each training and testing epoch, we generate the initial position guess $\hat{\mathbf{p}}_{\text{init}}$ by uniformly sampling from the interval $[\mathbf{p}_{\text{ECEF}} - \boldsymbol{\eta}, \mathbf{p}_{\text{ECEF}} + \boldsymbol{\eta}]$, where $\boldsymbol{\eta} = \eta \cdot [1, 1, 1]^\top$ is the vector-valued initialization range with a magnitude η that is the same along each direction. We use initial position guesses with randomly sampled noise added to the true position, in all our experiments, except without data augmentation, for training the network and validating/testing the trained network. The default value in the experimental validations is $\eta = 15\text{m}$, which is changed when studying the effect of different η values on the final position estimate.

Additionally, when evaluating the effectiveness of our data augmentation method, we compare our approach to a baseline without data augmentation. In the network without data augmentation, we use a fixed trajectory uniformly sampled from the interval $[\mathbf{p}_{\text{ECEF}} - \boldsymbol{\eta}, \mathbf{p}_{\text{ECEF}} + \boldsymbol{\eta}]$. Here, fixed implies that the samples are drawn once to generate the training and validation datasets and are not changed at any epoch during training.

2. Simulated Dataset

We create the simulated dataset by 1) generating smooth horizontal trajectories in the NED frame of reference, 2) converting the simulated trajectories to the ECEF frame of reference, and 3) simulating open sky GNSS measurements for each point along the trajectory.

We simulate trajectories to imitate real-world datasets, like the Android Raw GNSS Measurements Dataset (Fu et al., 2020), that are often confined to a limited geographical region and contain samples along vehicle trajectories. We simulate these trajectories based on the approach proposed by Mueller *et al.* in (Mueller et al., 2015). Note that our network performs snapshot position estimation, i.e., the correlation between samples in the trajectory has no impact on our experimental results.

To generate measurements for samples from the simulated trajectories, we use the standard pseudorange model (Morton et al., 2021) with the true position and clock states for each instance of data in the converted trajectories $\mathbf{p}_{\text{ECEF}}^{(i)}$. We do not consider any atmospheric effects or satellite clock biases in simulating the pseudorange measurements. Set $\bar{\mathcal{M}}^{(i)}$ represents the pairs of simulated pseudorange measurements and the corresponding satellite positions.

For each data instance, measurements are only simulated for satellites that are visible from $\mathbf{p}_{\text{ECEF}}^{(i)}$, determined using an elevation mask of 5° . Because we use an elevation mask to simulate the measurements, the number of measurements at each instance $M^{(i)}$ varies between 8-10 in our dataset. Additionally, we impose no constraints on the order of the simulated measurements.

We next describe experiments that utilize the simulated data to verify the validity of our approach. Additionally, we investigate the sensitivity of the DNN performance to the choice of measurement errors and the initialization range magnitude η .

Table 1: Mean absolute positioning error along the north, east and down directions in the position estimated using our approach across different types of error (Gaussian error and Gaussian + bias error) in the GNSS pseudorange measurements. In both scenarios, our approach reduces the positioning error over the baseline with random initialization by more than half the value.

Scenario	North (m)	East (m)	Down (m)
Initialization	7.5 ± 5.0	7.5 ± 5.0	7.5 ± 5.0
Gaussian error	2.6 ± 2.0	2.4 ± 1.8	2.2 ± 1.6
Gaussian + bias error	2.8 ± 2.1	2.6 ± 2.0	2.4 ± 1.8

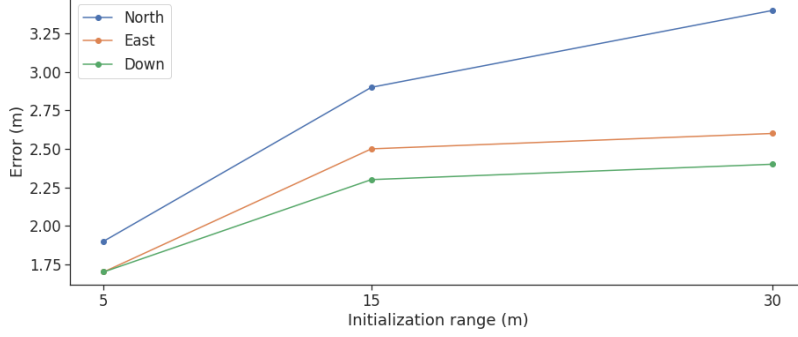


Figure 4: Sensitivity analysis over various initialization ranges along the north, east and down directions. The mean absolute error (MAE) in DNN-based position corrections increases when the initialization range is increased.

a) Verifying performance under different measurement errors

We verify the positioning performance of our DNN in our approach across two scenarios with different error profiles in the pseudorange measurements.

In the first scenario, simulated pseudoranges contain stochastic noise terms that follow a zero-mean Gaussian distribution with 6 m standard deviation. In the second scenario, we add bias errors along with the zero-mean Gaussian errors in the measurements. The bias errors are sampled from the interval [50, 200] m and are added to pseudoranges picked at random to mimic the effect of multipath and NLOS signals. The number of biased measurements at a time is sampled from a Poisson distribution with rate 1. In both scenarios, we use $\eta = 15$ m for generating the initial position guess $\hat{p}_{\text{init}}$. Because the DNN is not restricted by a prior measurement model, we hypothesize that the positioning error for the DNN should be unaffected by the noise scenarios, as long as the DNN encounters the same noise scenario during the training process.

To verify this hypothesis, we evaluate the mean absolute positioning error along the north, east and down directions respectively. For both scenarios, the positions estimated by applying corrections from our trained DNN exhibit positioning errors that are less than half the initial value, verifying that our proposed approach is effective in learning a function for positioning using GNSS measurements. These results are summarized in Table 1.

b) Comparing performance across different initial positions

Since the magnitude of the initialization range η determines the maximum initial positioning error, we expect it to have a significant effect on the positioning performance of the DNN. To investigate this, we evaluate the sensitivity of our approach to different choices of η for a scenario with zero-mean Gaussian errors in pseudorange measurements. We consider three different values of $\eta \in \{5 \text{ m}, 15 \text{ m}, 30 \text{ m}\}$ for training the DNN and compare the positioning performance of the resultant DNN, the results of which are shown in Fig. 4.

We observed that the positioning error along each of the north, east and down directions increases as we increase the value of η . However, this increase isn't linear and the difference between the positioning errors for $\eta = 15$ m and $\eta = 30$ m shows a less than linear growth. This indicates that while the positioning error of the DNN does depend on the magnitude of the initialization range η , the impact of η reduces as its magnitude increases.

We attribute the increase in the mean absolute error (MAE) on increasing the initialization range η to primarily two factors: 1) The network learns the maximum possible corrections based on the magnitude of the maximum error it sees in the training dataset. As a result, outputs for smaller values of η are restricted to smaller ranges, resulting in a smaller MAE. 2) On increasing η , the network must generalize to a larger set of possible inputs, which increases the overall error in the position estimate.

3. Android Raw GNSS Measurements Dataset

The Android Raw GNSS Measurements Dataset (Fu et al., 2020) consists of GNSS measurements collected using Android phones from multiple driving trajectories executed in the San Francisco Bay Area. This dataset has two components: 1) a training component and 2) a testing component. The training component is accompanied by high accuracy position estimates, collected using a NovAtel SPAN system, that we use as the ground truth position in our approach. Due to this availability of ground truth positions, we restrict ourselves to the training component because the ground truth provides a reference to both train and evaluate the DNN. Henceforth, we refer to this training component as the dataset for evaluating our approach. The GNSS measurements in each trajectory, referred to as a trace, include raw pseudoranges, atmospheric biases, satellite clock biases and satellite positions, from at least two Android phones. These measurements, including satellite positions, atmospheric

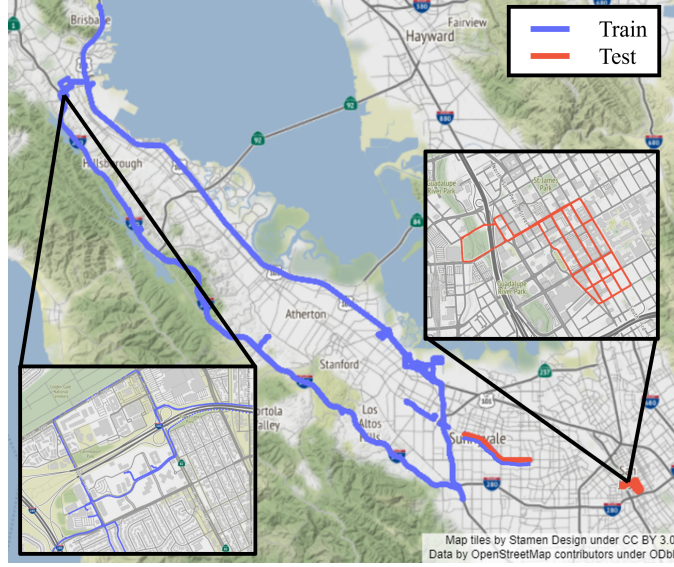


Figure 5: Traces from the Android Raw GNSS Measurements dataset. Only traces with ground truth position estimates available were used for our experimental evaluation. Traces belonging to the training/validation dataset are shown in blue, while traces belonging to the testing dataset are shown in red.

biases and satellite clock biases, are computed and provided in ‘Derived’ files in the dataset. We use these quantities without any modification or additional computations. We treat each unique phone-trace combination as an independent trajectory while validating our approach.

To create the set $\mathcal{M}^{(i)}$, for each data instance that is input to the DNN, we use measurements corresponding to GPS L1 signals and process the raw pseudoranges to remove errors that can be modeled. The corrected pseudorange $\rho_m^{(i)}$ is obtained from values present in the measurement dataset by

$$\rho_m^{(i)} = \bar{\rho}_m^{(i)} + B_m^{(i)} - b_m^{(i)} - I_m^{(i)} - T_m^{(i)}, \quad (23)$$

where $\bar{\rho}_m^{(i)}$ represents the raw pseudorange, $B_m^{(i)}$ the satellite clock bias, $b_m^{(i)}$ the inter-signal-ranging-bias, $I_m^{(i)}$ the modeled delay due to ionospheric effects and $T_m^{(i)}$ represents the modeled delay due to tropospheric effects. This process is repeated for all measurements $m \in \{1, \dots, M^{(i)}\}$ in all data instances $i \in \{1, \dots, N\}$, where $M^{(i)}$ is the number of measurements in the i th data instance and there are N data instances in the entire dataset.

In our experimental evaluation on the Android dataset, we split the dataset into three independent parts: 1) a training split ($\approx 75\%$ of the dataset), 2) a validation split ($\approx 10\%$ of the dataset), and a 3) testing split ($\approx 15\%$ of the dataset).

The first split divides the dataset into two parts: one for training/validation and another for testing. This division is performed on the trace level and the training/validation and testing dataset contain different traces, with all corresponding Android measurements from a particular trace associated with either the training/validation or testing dataset. The split between the training/validation and testing datasets is fixed and the same for all experiments in this work. The traces belonging to each dataset are plotted in Fig. 5. The additional split between the training and validation datasets is performed by randomly selecting a ratio of samples from the training/validation traces and using them to validate the network. Each split between the training and validation dataset is stochastic and changes from experiment to experiment.

As a result of the dataset split, the training dataset has 93195 samples, the validation dataset has 10355 samples, and the testing dataset has 16568 samples.

a) Performance Evaluation

We use the training split to train the DNN while the validation split is used to evaluate the DNN during training and ensure that it is learning successfully. We use the testing split to evaluate the performance of different variations of our approach and compare it to the WLS baseline.

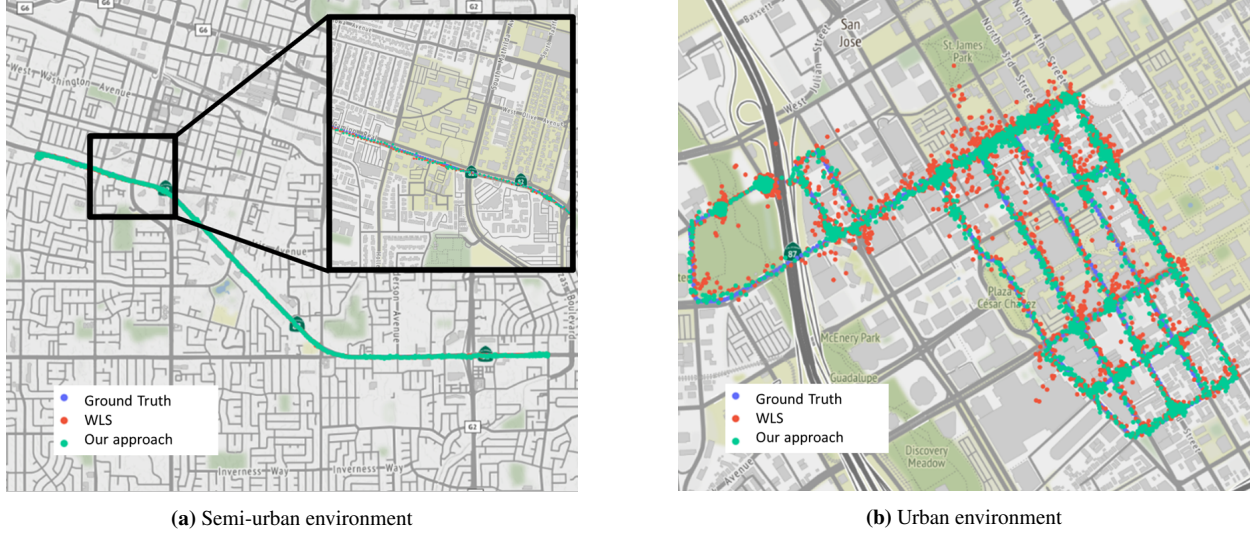


Figure 6: Example of localized trajectories on the Android Raw GNSS Measurements Dataset (Fu et al., 2020) for a) semi-urban and b) urban environment conditions. We visualize the positions estimated using our approach for $\eta = 15$ m and WLS along with the ground truth trajectory. The trajectory estimated using our approach is visually similar to WLS in the semi-urban environment and appears closer to the ground truth than WLS in the urban environment.

The WLS baseline position estimates are generated using the open-source goGPS implementation (Herrera et al., 2016). goGPS internally corrects pseudoranges by removing estimated atmospheric delays, satellite clock biases and other modelled biases. An elevation mask of 10° was applied on the received measurements and the remaining measurements were weighed using the default elevation-based weights from goGPS. The WLS output contained a 3D position estimate along with a clock bias estimate, of which we compare only the positions to those obtained by our proposed architecture.

We evaluate the performance of our proposed DNN with NED corrections and data augmentation using $\eta = 15$ m to our approach without augmentation, our approach with data augmentation using $\eta = 30$ m and the WLS baseline. This evaluation is performed on the entire testing dataset and our experiments show that our approach with $\eta = 15$ m performs the best out of all variations, both in terms of MAE, listed in Table 2 and CDF plots of the errors, shown in Fig. 7.

We also evaluated a network that predicted positions directly, instead of predicting corrections to an initial position. However, such an approach showed MAE in the order of 10^3 m along all directions and was not investigated further or compared to other methods.

Of the three variations of our method, that we evaluated, turning off the data augmentation has the least negative impact on the performance of the neural network. This difference is particularly noticeable in the North direction where the CDF curve deviates from the best case and an additional mean error of approximately 0.8 m is observed. The differences along the East and Down directions are not as evident, with an additional mean error of 0.15 m to 0.25 m and virtually indistinguishable CDF curves.

Similar to our observations from the simulated data, increasing the initialization range η increases the MAE and causes a perceptible drop in the CDF curve for same error values.

Performance of the WLS baseline is poorer than both networks initialized with $\eta = 15$ m in all three directions. However, the WLS baseline outperforms the network initialized with $\eta = 30$ m in the North and East directions while still performing poorly in the down direction.

This difference is further evidenced by a comparison of the error quantiles between our approach with $\eta = 15$ m, our approach with $\eta = 30$ m and the WLS baseline, as shown in Fig. 8. Our approach with $\eta = 15$ m outperforms the WLS baseline in all directions. However, with $\eta = 30$ m, our approach is only able to outperform WLS in the down direction. Similar to the simulated data, there is a strong correlation between the accuracy and the largest magnitude of the initial error, which is currently a limitation of this proposed work.

Fig. 8 also demonstrates that the network learns the largest magnitude of error in the training dataset and bounds the estimated position correction using this information. This also results in the improved performance of networks with smaller initialization ranges η which provide corrections with correspondingly smaller magnitudes. The network’s initial guess is always within a

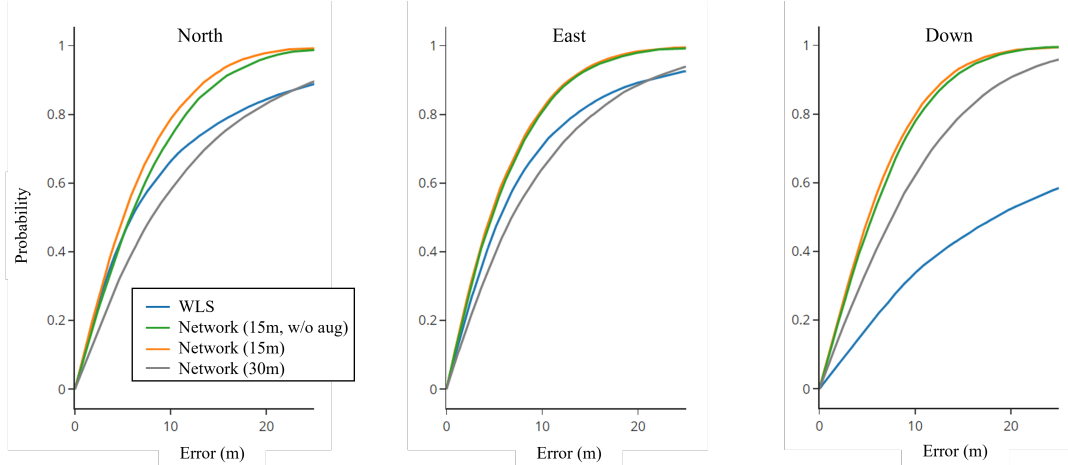


Figure 7: Cumulative distribution function (CDF) of the absolute error, on the test data, along the local North, East and Down directions for our approach with $\eta = 15\text{m}$, our approach without data augmentation, our approach with $\eta = 30\text{m}$ and the WLS baseline. We observe that our approach with $\eta = 15\text{m}$ outperforms all other variations and overbounds their corresponding CDFs. Our approach without data augmentation overbounds the WLS baseline as well. Our approach with $\eta = 30\text{m}$ is overbound by WLS for smaller values of error in the North and East directions but overbounds the WLS baseline in the Down direction.

certain range of the ground truth, because of which the network’s final estimate is also relatively closer to the ground truth solution. This results in our approach’s superior performance on data not seen during training, like the urban case visualized in Fig. 6. The semi-urban case visualized in Fig. 6 is similar to data encountered by the network during training and consequently, the network’s performs as expected in that case.

VI. CONCLUSION

In this work, we proposed an approach to use a deep neural network (DNN) with GNSS measurements to provide a position estimate. Our proposed approach is the first, to our knowledge, that works with GNSS measurements to provide outputs in the position domain.

To obtain a position estimate, we convert the traditional position estimation problem to that of estimating position corrections to an initial position guess using a DNN. Our proposed approach addresses the challenge of set-based GNSS inputs, that vary in number and order, by utilizing the set transformer in the DNN architecture. We proposed using pseudorange residuals and LOS vectors from the initial position guess as inputs and NED position corrections as outputs to the DNN. This particular choice of inputs and outputs improves the numerical conditioning of the DNN and provides a natural method to extend our approach to other global regions. Additionally, to reduce overfitting on training data and incentivize the DNN to learn a functional map between the measurements and position corrections, we developed a geometry-based data augmentation method.

We validated our proposed approach on both simulated and real-world data. Experiments performed on the simulated data showed that the position corrections provided by the DNN reduced the mean absolute localization error in each of the North, East and Down directions from the error in the initial position guess, indicating that the DNN effectively learns to solve the

Table 2: Mean absolute positioning error along the north, east and down directions in the estimate of the WLS baseline and variations of our approach (NED corrections + $\eta = 30\text{m}$, NED corrections + $\eta = 30\text{m}$ without data augmentation, and NED corrections + $\eta = 15\text{m}$). We observe that smaller initialization range results in smaller position estimate errors, data augmentation improves performance on the testing dataset and that final positioning errors are significantly less than those of WLS estimates in the Down direction for all cases.

Scenario	North (m)	East (m)	Down (m)
WLS baseline	11.6 ± 51.9	9.7 ± 38.7	36.4 ± 265.9
Our approach with $\eta = 30\text{m}$	11.1 ± 10.2	9.3 ± 8.5	9.3 ± 7.5
Our approach without data augmentation	7.1 ± 5.7	6.0 ± 5.1	6.6 ± 5.1
Our approach with $\eta = 15\text{m}$	6.4 ± 5.2	5.9 ± 5.0	6.2 ± 4.9

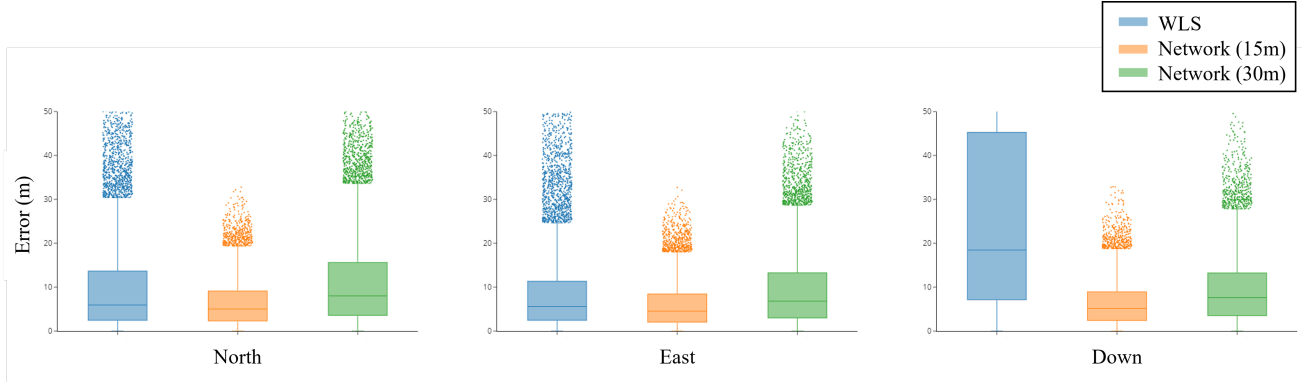


Figure 8: Localization error on the Android Raw GNSS Measurements Dataset (Fu et al., 2020) in the north, east and down directions respectively for WLS and our approach (initialization range $\eta = 15$ m and 30 m). The shaded regions cover the first quantile $Q1$ to the third quantile $Q3$, with the horizontal line representing the median. The vertical lines extend to a distance of $1.5 \cdot |Q3 - Q1|$ from $Q3$ and $Q1$ respectively. Points plotted beyond the vertical lines are samples at values greater than $Q3 + 1.5 \cdot |Q3 - Q1|$ and lesser than $Q1 - 1.5 \cdot |Q3 - Q1|$. Our approach with $\eta = 15$ m shows smaller localization error than WLS in all directions, while our approach with $\eta = 30$ m shows smaller localization error than WLS in the down direction.

positioning problem. Experiments on real-world data demonstrated that the performance of the DNN is sensitive to the error present in the initial position guess. Comparison of the absolute localization error to a weighted least squares (WLS) baseline showed that our approach outperforms WLS along the vertical direction when initialized with position errors within 15 m as well as 30 m. Our experimentation also validates that our data augmentation technique improves the network’s performance, when compared to a similar network without data augmentation.

This work validates that using DNNs for GNSS-based localization is a promising and interesting area of research. Our current approach is a snapshot method limited to using simple features. Additionally, both our training and testing datasets are entirely from the San Francisco Bay Area, which does not provide geographical diversity. In the future, we plan to validate our proposed method on diverse testing datasets collected from locations around the globe. We also plan to extend our approach to sequential position estimation while considering additional measurements such as signal-to-noise-ratio and Doppler. Furthermore, we are considering performing a more detailed parametric study to investigate the effect of hyperparameter values, the use of additional regularization methods and an iterative positioning correction approach similar to CMR Net (Cattaneo et al., 2019). Our proposed work is also limited by its reliance on close initial guesses and the sensitivity to initialization ranges, which we will also address in future work.

ACKNOWLEDGEMENTS

Some of the computing for this project was performed on the Sherlock cluster. We would like to thank Stanford University and the Stanford Research Computing Center for providing computational resources and support that contributed to this research.

REFERENCES

- Cattaneo, D., Vaghi, M., Ballardini, A. L., Fontana, S., Sorrenti, D. G., & Burgard, W. (2019). CMRNet: Camera to LiDAR-Map Registration. *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, 1283–1289. <https://doi.org/10.1109/ITSC.2019.8917470>
- Choy, C., Dong, W., & Koltun, V. (2020). Deep Global Registration. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2511–2520. <https://doi.org/10.1109/CVPR42600.2020.00259>
- Fu, G. M., Khider, M., & van Diggelen, F. (2020). Android Raw GNSS Measurement Datasets for Precise Positioning. *Proceedings of the 33rd International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS+ 2020)*, 1925–1937. <https://doi.org/10.33012/2020.17628>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT press. <https://dl.acm.org/doi/abs/10.5555/3086952>
- Herrera, A. M., Suhandri, H. F., Realini, E., Reguzzoni, M., & de Lacy, M. (2016). Gogps: Open-source matlab software. *GPS solutions*, 20(3), 595–603.
- Hsu, L.-T. (2017). GNSS Multipath Detection Using a Machine Learning Approach. *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, 1–6. <https://doi.org/10.1109/ITSC.2017.8317700>

- J. McKeown, J., Stella, F., & Hall, G. (1997). Some numerical aspects of the training problem for feed-forward neural nets. *Neural Networks*, 10(8), 1455–1463. Retrieved April 22, 2022, from [https://doi.org/10.1016/S0893-6080\(97\)00015-4](https://doi.org/10.1016/S0893-6080(97)00015-4)
- Kanhere, A. V., Gupta, S., Shetty, A., & Gao, G. (2021). Improving GNSS Positioning using Neural Network-based Corrections. *Proceedings of the 34th International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS+ 2021)*. <https://doi.org/10.33012/2021.17999>
- Kendall, A., Grimes, M., & Cipolla, R. (2015). PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization. *2015 IEEE International Conference on Computer Vision (ICCV)*, 2938–2946. <https://doi.org/10.1109/ICCV.2015.336>
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*. <https://hdl.handle.net/11245/1.505367>
- Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., & Teh, Y. W. (2019). Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. In K. Chaudhuri & R. Salakhutdinov (Eds.), *Proceedings of the 36th international conference on machine learning* (pp. 3744–3753). PMLR. <https://proceedings.mlr.press/v97/lee19d.html>
- Mikhailov, N. V., & Nikandrov, A. V. (2012). Identification and Mitigation of Multipath in GNSS Receivers Using Cluster Analysis Methods. *Gyroscopy and Navigation*, 3(1), 20–27. <https://doi.org/10.1134/S2075108712010105>
- Morton, Y. J., van Diggelen, F., Spilker Jr, J. J., Parkinson, B. W., Lo, S., & Gao, G. (2021). *Position, Navigation, and Timing Technologies in the 21st Century: Integrated Satellite Navigation, Sensor Systems, and Civil Applications*. John Wiley & Sons.
- Mueller, M. W., Hehn, M., & D’Andrea, R. (2015). A Computationally Efficient Motion Primitive for Quadcopter Trajectory Generation. *IEEE Transactions on Robotics*, 31(6), 1294–1310. <https://doi.org/10.1109/TRO.2015.2479878>
- Munin, E., Blais, A., & Couellan, N. (2020). Convolutional Neural Network for Multipath Detection in GNSS Receivers. *2020 International Conference on Artificial Intelligence and Data Analytics for Air Transportation (AIDA-AT)*, 1–10. <https://doi.org/10.1109/AIDA-AT48540.2020.9049188>
- Peretroukhin, V., & Kelly, J. (2018). DPC-Net: Deep Pose Correction for Visual Localization. *IEEE Robotics and Automation Letters*, 3(3), 2424–2431. <https://doi.org/10.1109/LRA.2017.2778765>
- Reisdorf, P., Pfeifer, T., Breßler, J., Bauer, S., Weissig, P., Lange, S., Wanielik, G., & Protzel, P. (2016). The Problem of Comparable GNSS Results—An Approach for a Uniform Dataset with Low-cost and Reference Data. *Proc. of Intl. Conf. on Advances in Vehicular Systems, Technologies and Applications (VEHICULAR)*.
- Savas, C., & Doyis, F. (2019). Multipath Detection based on K-means Clustering. *Proceedings of the 32nd International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS+ 2019)*, 3801–3811. <https://doi.org/10.33012/2019.17028>
- Skianis, K., Nikolentzos, G., Limnios, S., & Vazirgiannis, M. (2020). Rep the Set: Neural Networks for Learning Set Representations. In S. Chiappa & R. Calandra (Eds.), *Proceedings of the twenty third international conference on artificial intelligence and statistics* (pp. 1410–1420). PMLR. <https://proceedings.mlr.press/v108/skianis20a.html>
- Soelch, M., Akhundov, A., van der Smagt, P., & Bayer, J. (2019). On Deep Set Learning and the Choice of Aggregations. In I. V. Tetko, V. Kůrková, P. Karpov, & F. Theis (Eds.), *Artificial neural networks and machine learning – icann 2019: Theoretical neural computation* (pp. 444–457). Springer International Publishing. https://doi.org/10.1007/978-3-030-30487-4%5C_35
- Sünderhauf, N., Brock, O., Scheirer, W., Hadsell, R., Fox, D., Leitner, J., Upcroft, B., Abbeel, P., Burgard, W., Milford, M., & Corke, P. (2018). The Limits and Potentials of Deep Learning for Robotics. *The International Journal of Robotics Research*, 37(4-5), 405–420. <https://doi.org/10.1177/0278364918770733>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is All You Need. *Advances in neural information processing systems*, 5998–6008.
- Wang, S., Clark, R., Wen, H., & Trigoni, N. (2017). DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks. *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2043–2050. <https://doi.org/10.1109/ICRA.2017.7989236>
- Wen, W., Zhou, Y., Zhang, G., Fahandezh-Saadi, S., Bai, X., Zhan, W., Tomizuka, M., & Hsu, L.-T. (2020). UrbanLoco: A Full Sensor Suite Dataset for Mapping and Localization in Urban Scenes. *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2310–2316. <https://doi.org/10.1109/ICRA40945.2020.9196526>
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., & Smola, A. J. (2017). Deep Sets. *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 3394–3404. <https://dl.acm.org/doi/10.5555/3294996.3295098>
- Zhang, G., Xu, P., Xu, H., & Hsu, L.-T. (2021). Prediction on the Urban GNSS Measurement Uncertainty Based on Deep Learning Networks With Long Short-Term Memory. *IEEE Sensors Journal*, 21(18), 20563–20577. <https://doi.org/10.1109/JSEN.2021.3098006>
- Zhu, N., Marais, J., Betaille, D., & Berbineau, M. (2018). GNSS Position Integrity in Urban Environments: A Review of Literature. *IEEE Transactions on Intelligent Transportation Systems*, 19(9), 2762–2778. <https://doi.org/10.1109/TITS.2017.2766768>